

Объектно-ориентированное программирование на примере языка программирования Python.

Долгих А.А.
ИИР ПИИ 1 курс 25945

1. История и эволюция подхода

Хотя современное программирование и сейчас опирается на математические модели 1930-х годов, такие как машина Тьюринга и лямбда-исчисление, объектно-ориентированный подход возник значительно позже как ответ на кризис сложности программных систем [1]. Первым языком, в котором формально появились классы и иерархии наследования, стал Simula 67, предназначенный для моделирования физических процессов [1].

Однако само понятие «объектно-ориентированное программирование» было предложено Аланом Кеєм в 1967 году, а затем реализовано при разработке языка Smalltalk в начале 1970-х годов [4]. Кей представлял объекты как «биологические клетки», которые взаимодействуют исключительно через обмен сообщениями, скрывая свою внутреннюю структуру [4]. В дальнейшем индустрия разделилась на две ветви: «чистое» ООП (Smalltalk) и ветвь на основе абстрактных типов данных (C++, Java), которая в итоге стала доминирующей в промышленной разработке [1, 5].

2. Введение в классы

Объектно-ориентированное программирование (ООП) — это парадигма, в которой программа рассматривается как сеть взаимодействующих объектов, каждый из которых является экземпляром определенного класса [2]. В этой модели класс выступает в роли «чертежа» или шаблона, а объект (экземпляр) — его конкретной реализации в памяти [2].

В языке Python классы обладают тремя особенностями, отличающими их от модулей:

1. **Множество экземпляров:** возможность создавать неограниченное количество объектов с собственным состоянием [2].
2. **Настройка через наследование:** механизм расширения функционала без переписывания базового кода [1, 2].
3. **Перегрузка операций:** использование специальных методов для интеграции объектов во встроенные механизмы языка [2].

3. Базовые принципы ООП

Классическое ООП строится на нескольких ключевых концепциях, позволяющих эффективно декомпозировать сложные задачи.

3.1. Абстракция

Абстракция — это принцип выделения только значимых характеристик объекта и игнорирование несущественных деталей реализации [1]. Она позволяет разработчику оперировать высокоуровневыми концепциями, не погружаясь в технические подробности того, как именно устроены данные «под капотом» [1, 5].

3.2. Инкапсуляция

Инкапсуляция подразумевает объединение данных (атрибутов) и алгоритмов их обработки (методов) в единую сущность — объект [1]. Основная цель заключается в изоляции внутреннего состояния объекта от внешнего мира, что гарантирует целостность данных и упрощает сопровождение кода [1, 5].

3.3. Наследование

Наследование позволяет создавать новые классы на основе существующих, перенимая их свойства и поведение для устранения дублирования кода [1]. В Python для вызова методов родителя используется функция `super()`, которая критически важна для корректной работы в сложных иерархиях [2].

3.4. Полиморфизм

Это способность программы использовать объекты разных классов с одинаковым интерфейсом без знания их конкретного типа [1]. В Python полиморфизм часто опирается на концепцию утиной типизации (*duck typing*), где поведение объекта важнее его положения в иерархии классов [2].

4. Объекты и специальные методы

Python предоставляет богатый набор специальных (или «магических») методов, которые начинаются и заканчиваются двойным подчеркиванием. Они управляют жизненным циклом объектов и позволяют им реагировать на стандартные операции языка [2].

- **`__new__(cls, ...)`**: Метод, отвечающий за фактическое выделение памяти и создание экземпляра перед его инициализацией [2].
- **`__init__(self, ...)`**: Конструктор, вызываемый автоматически для инициализации атрибутов созданного объекта [2].
- **`__str__(self)`**: Определяет удобочитаемое строковое представление объекта для функций `print()` и `str()` [2].
- **`__len__(self)`**: Позволяет объекту возвращать свою длину при вызове встроенной функции `len()` [2].
- **`__getitem__(self, key)`**: Реализует доступ по индексу `[]`, позволяя объекту имитировать поведение списка или словаря [2].

5. Инкапсуляция в Python

Хотя Python не запрещает доступ к данным на уровне виртуальной машины (как C++ или Java), в нем реализованы механизмы и соглашения, обеспечивающие защиту данных [2].

1. **Защищенные атрибуты (`_name`)**: Это соглашение внутри сообщества, указывающее, что атрибут является внутренним. Доступ к нему извне технически возможен, но считается плохой практикой [2].
2. **Приватные атрибуты (`__name`)**: Включают механизм искажения имен (**`name mangling`**), автоматически заменяя имя на `_ClassName__name`, что предотвращает случайные конфликты или переопределение в подклассах [2].
3. **Свойства (`@property`)**: Декоратор, превращающий метод в геттер. Вместе с сеттером он позволяет внедрять логику валидации при изменении данных, не меняя публичный интерфейс класса [2].

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    @property
    def balance(self):
        return self.__balance
```

6. Принципы проектирования: SOLID, KISS, YAGNI

Для создания долгоживущих систем были выработаны принципы, предотвращающие архитектурную деградацию.

6.1. SOLID

Пять принципов, сформулированных Робертом Мартином [5]:

- SRP (Single Responsibility): Класс должен иметь только одну причину для изменения [5].
- OCP (Open/Closed): Программные сущности должны быть открыты для расширения, но закрыты для модификации [5].
- LSP (Liskov Substitution): Объекты подтипа должны быть взаимозаменяемы с объектами родительского класса без нарушения корректности программы [5].
- ISP (Interface Segregation): Клиенты не должны зависеть от интерфейсов и методов, которые они не используют [5].
- DIP (Dependency Inversion): Модули верхних уровней не должны зависеть от модулей нижних уровней; оба типа модулей должны зависеть от абстракций [5].

6.2. KISS и YAGNI

Принцип KISS (Keep It Simple, Stupid) призывает выбирать максимально простые и легко читаемые решения [5]. YAGNI (You Aren't Gonna Need It) предостерегает от преждевременной оптимизации и добавления функционала «на будущее», который прямо сейчас не требуется [1, 5].

7. Множественное наследование и MRO

Python поддерживает наследование от нескольких родителей, что порождает «проблему ромба» — неоднозначность при поиске методов. Для ее решения используется алгоритм C3-линеаризации, определяющий порядок разрешения методов (MRO — Method Resolution Order) [3].

Основные правила MRO:

- Поиск ведется снизу вверх (от потомка к предкам) [3].
- Соблюдается локальное старшинство (порядок перечисления классов при объявлении) [3].
- Линеаризация должна быть монотонной: порядок следования предков в подклассе не должен противоречить их порядку в родительских классах [3].

8. Список литературы

1. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Паттерны объектно-ориентированного проектирования. – СПб.: Питер, 2021. – 448 с.
2. Белоцерковский В. А. ООП в Python. Создание собственных классов. Лекция 10–11. – НГУ, 2025.
3. Порядок разрешения методов в Python [Электронный ресурс]. – URL: <https://habr.com/ru/articles/62203/> (дата обращения 18.05.2026).
4. Доктор Алан Кей о смысле «объектно-ориентированного программирования» [Электронный ресурс]. – URL: <https://habr.com/ru/articles/946868/> (дата обращения 18.05.2026).
5. Martin R. Clean Architecture: A Craftsman's Guide to Software Structure and Design. – Prentice Hall, 2017. (Или рус. пер.: Мартин Р. Чистая архитектура. – СПб.: Питер, 2018).